

哈尔滨工业大学

编译原理实验报告（一）

姓名	曹云涵
学号	2021113258
学院	未来技术学院
教师	单丽莉

2024 年 4 月

一、实验目的

1. 巩固对词法分析与语法分析的基本功能和原理的认识
2. 能够应用自动机的知识进行词法分析和语法分析
3. 理解并处理词法分析与语法分析中的异常和错误

二、实验环境

Windows11-WSL- Ubuntu_22.04
GCC 11.4.0
Flex 2.6.4
Bison 3.8.2

三、实验过程

1. 本次实验用到的数据结构

在本次实验中，通过使用语法分析树来进行语法分析，采纳了一种特别的树结构。为了有效利用存储空间，这里使用了一种特殊的多叉树二叉树表示法。在此方法中，左孩子节点表示子节点，而右孩子节点则表示兄弟节点。该表示法使得可以通过链表存储树结构，既节约了存储空间又能准确知道树的大小。

树中的每个节点代表一个语法分析单元，并通过联合体来表示节点信息。由于不同的语法单元可能有不同数量的子节点，因此选用了 stdarg 库中的 va_list 数据结构来支持可变数量的参数。

设计验证无误后，下一步是对代码进行输出，这需要对语法分析树进行遍历。因此，还需在树结构基础上实现深度优先遍历算法，完成整个数据结构设计。

```
struct Parsetree
{
    int line;
    char *Token;
    int isleaf;
    union
    {
        char* Id_Type;
        int intval;
        float floatval;
    };
    struct Parsetree *firstchild,*nxttbro;
};

static void dfs(Treenode now, int depth)
{
    if (now==NULL||now->Token==NULL) return;
    printf("%s",2*depth,"");
    printf("%s",now->Token);
    if((!strcmp(now->Token,"ID"))||(!strcmp(now->Token,"TYPE")))
        printf(": %s",now->Id_Type);
    else if(!strcmp(now->Token,"INT"))
        printf(": %d",now->intval);
    else if(!strcmp(now->Token,"FLOAT"))
        printf(": %f",now->floatval);
    else if(!now->isleaf)
        printf(" (%d)",now->line);
    puts("");
    dfs(now->firstchild,depth+1);
    dfs(now->nxttbro,depth);
}
```

图 1：数据结构 Parsetree 的核心代码

2. 基于 flex 的词法分析过程

通过 lexical.l 文件的编写实现对于词法的分析，在这个过程中比较重要的一步就是书写特定词的正则表达式。较为复杂的三个部分分别是 INT、FLOAT 和 ID。对于其它的部分，只需将名称与对应的符号进行匹配即可。在最终的词法分析中，我会将 C--程序拆分成一个一个的词进行输出。

```
INT [1-9][0-9]*|0
FLOAT {INT}\.[0-9]+
ID [a-zA-Z_][a-zA-Z_0-9]*

LC "{"
RC "}"
LB "["
RB "]"
LP "("
RP ")"
```

图2：Flex 代码的编写（部分）

同样，程序也会针对一些特定的错误词法设计相对应的错误解决办法，在本实验中将这种错误记为了 A 类错误。

```
{INT}+{ID} {hasFault = 1; printf("Error type A at Line %d: Illegal ID '%s'\n", yylineno, yytext); }
\.{INT} {hasFault = 1; printf("Error type A at Line %d: Illegal float number '%s'\n", yylineno, yytext); }
. {hasFault = 1; printf("Error type A at Line %d: Mysterious characters '%s'\n", yylineno, yytext); }
%%
```

图3：错误的处理形式

基于编写的 lexical.l 以及稍后编写的主.c 即可完成词法分析这一部分。

3. 基于 bison 的语法分析过程

编写 syntax.y 文件是进行语法分析的关键步骤之一。首先，需要明确定义语法分析过程中的终结符和非终结符，这是因为整个语法分析是基于自动机理论实现的。

语法分析的一个重要环节是错误处理。在 syntax.y 文件中，首先要列举出所有正确的语法规则并定义其后续操作。对于不符合这些规则的情况，可以通过使用 error 单元来统一处理。这样，语法分析器能够有效地识别和处理错误情况，保证语法分析的准确性和稳定性。

```
%type <node> Program ExtDefList ExtDef ExtDecList // High-level Definitions %token <node> INT FLOAT ID
%type <node> Specifier StructSpecifier OptTag Tag // Specifiers %token <node> SEMI COMMA
%type <node> VarDec FunDec VarList ParamDec // Declarators %token <node> ASSIGNOP RELOP
%type <node> CompSt StmtList Stmt // Statements %token <node> PLUS MINUS STAR DIV
%type <node> DefList Def Dec Declist // Local Definitions %token <node> LP RP LB RB LC RC
%type <node> Exp Args // Expressions %token <node> TYPE STRUCT RETURN
%token <node> IF ELSE WHILE
```

图4：自定义的终结符和非终结符

```
ExtDef : Specifier ExtDecList SEMI {$$ = newnode(@$.first_line, "ExtDef", 3, $1, $2, $3);}
| Specifier SEMI {$$ = newnode(@$.first_line, "ExtDef", 2, $1, $2);}
| Specifier FunDec CompSt {$$ = newnode(@$.first_line, "ExtDef", 3, $1, $2, $3);}
| error SEMI {hasFault = 1;}
```

图5：syntax.y 代码（部分）

在处理到一个有效节点时，首先要创建一个树节点。接着，将节点信息、节点的子节点数量和子节点的编号都传递给创建节点的函数。当前节点是否为终结符的判断依据是其子节点的数量：如果子节点数量为 0，则是终结符；否则，不是。对于终结符，需要提取相关信息；而对于非终结符，只需继续处理其子节点即可。这一部分的代码实现在`tree\_node.h`中体现。

```
if (amount==0)
{
    rt->isleaf=1;
    rt->firstchild=NULL;
    if(!strcmp(TOKEN,"ID")){if(!strcmp(TOKEN,"TYPE"))
    {
        char *str;
        str=(char*)malloc(sizeof(char)*40);
        strcpy(str,yytext);
        rt->id_type=str;
    }
    else if(!strcmp(TOKEN,"INT"))
        rt->intval=atoi(yytext);
    else if(!strcmp(TOKEN,"float"))
        rt->floatval=atof(yytext);
}
else
{
    va_list list;
    va_start(list,amount);
    ch=va_arg(list,tree_node);
    rt->firstchild=ch;
    if (amount==1)
    {
        for (int i=1;i<=amount-1;i++)
        {
            ch->nextbro=va_arg(list,tree_node);
            ch->nextbro->nextbro=NULL;
        }
        va_end(list);
    }
    return rt;
}
```

图6：处理节点的代码

4. 实验的操作流程

1. 编写词法分析文件（lexical.l）：首先编写包含词法规则的 lexical.l 文件。
2. 利用 Flex 编译词法文件：使用 Flex 工具编译 lexical.l，生成 lex.yy.c 文件。

3. 编写主函数（main.c）：创建 main.c 文件，它将与 lex.yy.c 一起编译。
4. 编译生成词法分析工具（scanner）：将 main.c 和 lex.yy.c 文件共同编译，生成词法分析工具，命名为 scanner。
5. 编写语法分析文件（syntax.y）：接着编写包含语法规则的 syntax.y 文件，并在其中引用 lex.yy.c。
6. 利用 Bison 编译语法文件：使用 Bison 工具编译 syntax.y，产生 syntax.tab.c 和 syntax.tab.h 文件。
7. 更新词法分析文件：在 lexical.l 中引用 syntax.tab.h，然后重新使用 Flex 编译，得到新的 lex.yy.c 文件。
8. 编写主函数（main.c）：创建新的 main.c 文件，用于语法分析工具。
9. 编译生成语法分析工具（parser）：将更新后的 main.c 和 syntax.tab.c 共同编译，生成语法分析工具，命名为 parser。

5. 实验的运行结果（仅展示部分）



图 7：scanner 运行结果（左为有错误 右为无错误）

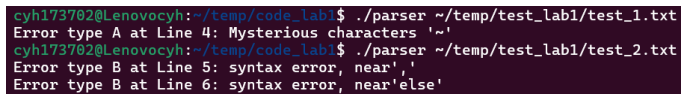


图 8：parser 分析得到语法错误的情况

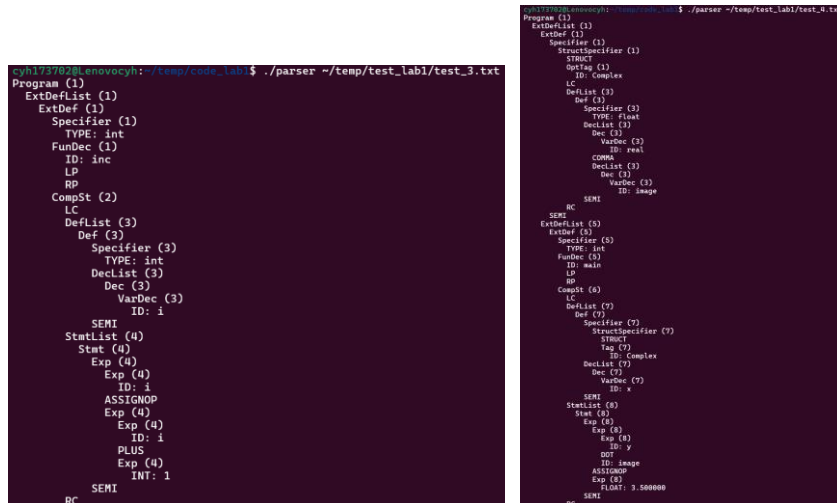


图 9：parser 分析正确代码的情况

四、实验心得

在这次编译原理的实验中，我深入了解了词法分析和语法分析的基本功能与原理。通过实际编写 lexical.l 和 syntax.y 文件，我应用了自动机的知识，实现了这两个关键过程。体会到了在处理语法和词法错误时的挑战，以及如何通过细致的编码和测试来应对这些问题。

这次实验也加深了我对理论知识和实践应用之间联系的理解。尽管遇到了一些难点，比如编写复杂的正则表达式和处理多叉树的二叉树表示法。实际编写代码和测试程序的过程和测试的过程中遇到的许多问题都加深了我对于编译的理解。